

EngageIQ Technical Brief

Architecture

EngageIQ is a full-stack Python application that discovers and ranks public engagement opportunities from GitHub, GH Archive, Reddit-style communities, and Hacker News. The local demo runs from `code/app.py` using Python's built-in HTTP server. The data layer began as a deterministic 10,500-record offline snapshot across all 15 required domains; after live ingestion tests, the current local snapshot contains 76,972 unique records. Optional live collectors in `code/ingest_live.py` and `code/engageiq/collectors.py` normalize public API records into the same schema.

The pipeline is: multi-source ingestion schema -> deduplication by stable id -> text representation -> vector retrieval -> composite engagement scoring -> diversity-aware re-ranking -> adaptive feedback updates -> trend analytics -> exportable brief.

BAX-423 Techniques

The first course technique is semantic embedding retrieval. EngageIQ represents user profiles and opportunity records as Sentence-BERT dense vectors using `all-MiniLM-L6-v2`, then retrieves candidates by cosine similarity. The dense matrix is cached locally so the dashboard can reuse the embeddings after the first build. If the model is unavailable, TF-IDF cosine retrieval remains as a fallback so the app still runs in offline grading environments. Dense embeddings help match noisy community text from GitHub, Reddit-style posts, GH Archive, and Hacker News to user goals even when exact keywords differ.

The second technique is multi-stage recommendation and ranking. Candidate generation retrieves the most relevant records, then a scoring layer combines semantic relevance, community health, visibility, freshness, effort fit, and learned feedback affinity. A final re-ranking step prevents narrow source/domain repetition.

The third supporting technique is adaptive learning from feedback. The app records engage, skip, and bookmark signals as tag-level weights, then the ranking model updates future affinity scores. Real dashboard feedback is persisted to `data/feedback_events.json` and replayed on startup, so learned preferences survive restarts. `code/evaluate_personas.py` and the dashboard simulation demonstrate 60 feedback rounds and report Precision@10 before and after adaptation.

The dashboard is deterministic for a given profile, dataset, filter set, and feedback state. Rankings change when the user switches personas, edits a custom profile, changes source filters, records feedback, or appends live data. Custom profiles can be saved locally in the browser and then selected from the persona dropdown.

Persona Tests

Sofia ML Student: Top recommendations emphasize Machine Learning and Beginner Coding, with GitHub good-first-issue opportunities and C++/Rust avoidance.

David DevOps Engineer: Top recommendations emphasize DevOps/K8s and Cloud APIs, with infrastructure communities and discussion-oriented opportunities.

Lina Data Journalist: Ranking boosts freshness and growth velocity, and the analytics tab shows trending domains, communities, and recent volume.

Raj Startup Founder: Recommendations prioritize Developer Tools, APIs, CLI tooling, and startup-relevant discussion threads; skip feedback reduces low-fit records.

With the SBERT dense embedding cache active, the benchmark returns Precision@10 of 1.00 for Sofia, Lina, and Raj, while David improves from 0.90 to 1.00 after simulated feedback.

Limitations

The current build is optimized for a reliable local and offline grading demo. The offline snapshot is synthetic but schema-compatible with public API collectors, and live GitHub/GH Archive/Hacker News ingestion has been added through a separate script. A production version should add scheduled ingestion jobs, approved Reddit PRAW collection, authenticated user accounts, and deployment to a public URL. The app intentionally does not track private user activity on GitHub, Reddit, or Hacker News; it only uses public opportunity records, local profile input, and in-app feedback.